

— MODULE 04

JavaScript Async: Timing and Data

Fetching data, handling delays, and building interfaces that don't freeze while they wait.

THE SYNCHRONOUS PROBLEM

JavaScript runs on a single thread. It executes one line of code at a time. If a task (like downloading a large file) takes 5 seconds, the entire webpage freezes for 5 seconds. We need a way to start a task and move on.

What is Asynchronous Code?

Asynchronous code allows JavaScript to start a long-running task, move on to execute other code, and then handle the task's result when it finally finishes.

THE RESTAURANT METAPHOR

Imagine ordering food. You don't stand at the counter blocking everyone else until your food is ready (synchronous). Instead, you take a buzzer, sit down, and do other things until the buzzer goes off (asynchronous).

THE EVENT LOOP

JavaScript uses an "Event Loop" to manage this. It hands off tasks like network requests or timers to the browser, continues running your main script, and puts the finished tasks back in line to be handled later.

A SIMPLE EXAMPLE: SETTIMEOUT

```
console.log('1. Order placed');  
  
// This task is handed off to the browser  
setTimeout(function() {  
  console.log('3. Food is ready!');  
}, 2000);  
  
// JS keeps running immediately  
console.log('2. Sitting down to wait');
```

The Old Way: Callbacks

Originally, we handled asynchronous tasks by passing a function (a callback) to be executed when the task finished. This worked, until things got complicated.

WHAT IS A CALLBACK?

A callback is simply a function passed as an argument to another function, meant to be called later (e.g., the function inside `addEventListener`).

"CALLBACK HELL"

If you need to perform several asynchronous tasks in a specific order, you have to nest callbacks inside callbacks. This creates the "Pyramid of Doom," making code incredibly hard to read and debug.

THE PYRAMID OF DOOM

```
getUser('vol', function(user) {  
  getProfile(user.id, function(profile) {  
    getPosts(profile.id, function(posts) {  
      getComments(posts[0].id,  
function(comments) {  
      // It just keeps moving right...  
      console.log(comments);  
    });  
  });  
});  
});
```

The Modern Way: Promises

A Promise is an object representing the eventual completion (or failure) of an asynchronous operation. Instead of passing callbacks *into* a function, the function returns a Promise that you attach callbacks *to*.

1. PENDING

The initial state. The asynchronous operation has started but has not yet completed. We are waiting for the result.

2. FULFILLED

The operation completed successfully. The Promise now holds the resulting data, which we access using `.then()`.

3. REJECTED

The operation failed (e.g., network error). The Promise holds the error reason, which we catch using `.catch()`.

FLATTENING THE PYRAMID

```
// Promises allow us to chain operations vertically instead of nesting them
getUser('vollo')
  .then(user => getProfile(user.id))
  .then(profile => getPosts(profile.id))
  .catch(error => console.error('Something went wrong!', error));
```

async / await

Modern JavaScript provides a way to write asynchronous code that reads top-to-bottom, just like synchronous code, without the mess of nested callbacks.

THE ASYNC KEYWORD

Placing `async` before a function tells JavaScript that this function will handle asynchronous operations and returns a Promise.

THE AWAIT KEYWORD

Used *only* inside an `async` function. It pauses the execution of that function until the Promise resolves, **without** freezing the rest of the webpage.

FLATTENING THE PYRAMID

```
// Modern, readable
async function loadUserProfile(userId) {
  const user = await getUser(userId);
  const profile = await getProfile(user.id);
  console.log(profile);
  return profile;
}
```

The Fetch API

Modern JavaScript provides the `fetch()` function to request data from servers or APIs without reloading the page.

HOW IT WORKS

You call `fetch()` with a URL. Because network requests take time, `fetch()` returns a Promise that resolves when the server responds.

THE TWO-STEP PROCESS

The initial response contains headers and status. To read data (like JSON), call a second async method on the response.

FETCHING DATA WITH ASYNC/AWAIT

```
async function getUserData() {  
  // 1. Make the network request  
  const response = await  
    fetch('https://api.example.com/users/1');  
  
  // 2. Extract the JSON data  
  const data = await response.json();  
  
  console.log(data.name);  
}  
  
getUserData();
```

Handling JSON

When a server sends data back, it usually arrives as a raw text string formatted as JSON. We must parse it into a usable JavaScript object.

WHAT IS JSON?

JavaScript Object Notation (JSON) is a lightweight data-interchange format. It looks exactly like a JavaScript object, but it's transmitted as pure text over the network.

THE .JSON() METHOD

The `fetch()` response object has a built-in `.json()` method. Because parsing a large text string can take time, this method is also asynchronous and returns a Promise.

PARSING THE RESPONSE

```
async function getUser() {  
  const response = await  
    fetch('/api/user');  
  
  // Parse the text into a JS object  
  const data = await response.json();  
  
  // Now we can use it!  
  console.log(data.name);  
}
```

Handling Errors

Network requests can fail. Servers go down, URLs change, or users lose their connection. We must handle these failures gracefully so our app doesn't crash.

TRY / CATCH

Wrap risky code inside a `try` block. If any error occurs (like a network failure), execution jumps to `catch` where you can handle it safely.

THE FETCH QUIRK

`fetch()` only throws when the network fails. A 404 or 500 still resolves; you must check `response.ok`.

ROBUST FETCH REQUEST

```
async function getSafeData() {
  try {
    const response = await fetch('/api/data');
    if (!response.ok) throw new Error(`HTTP
error: ${response.status}`);
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error('Fetch failed:',
error.message);
  }
}
```


Local Storage

Save small amounts of data directly in the user's browser. This data survives page refreshes and even browser restarts, making it perfect for user preferences.

KEY-VALUE PAIRS

Data is saved using a unique name (the key) and its corresponding data (the value). It acts like a simple dictionary attached to the browser tab.

STRINGS ONLY

Local Storage can **only** save strings. To save complex data like arrays or objects, you must convert them using `JSON.stringify()` first.

SAVING AND LOADING OBJECTS

```
// 1. Saving an object (Convert to string first)
const theme = { mode: 'dark', fontSize: '16px' };
localStorage.setItem('userTheme', JSON.stringify(theme));

// 2. Retrieving the object (Parse back to JS object)
const savedTheme = localStorage.getItem('userTheme');
if (savedTheme) {
  const parsedTheme = JSON.parse(savedTheme);
  console.log(parsedTheme.mode); // "dark"
}
```

Session Storage

Sometimes you only want to store data temporarily. Session Storage works exactly like Local Storage, but with a much shorter lifespan.

LOCAL STORAGE (PERSISTENT)

Data saved in `localStorage` stays in the browser forever, until the user explicitly clears their browser data or your code deletes it. Great for user preferences or themes.

SESSION STORAGE (TEMPORARY)

Data saved in `sessionStorage` is cleared the moment the user closes the specific browser tab or window. Perfect for sensitive data or temporary form states.

IDENTICAL API SYNTAX

```
// Save data for this session only  
sessionStorage.setItem('draftPost',  
  'Hello...');
```

```
// Retrieve it (e.g., after a page  
refresh)
```

```
const draft =  
sessionStorage.getItem('draftPost');
```

```
// Remove a specific item  
sessionStorage.removeItem('draftPost');
```

```
// Clear everything in session storage  
sessionStorage.clear();
```